How Our Project Works

Learning to Initialize Viscosity Estimation from Sparse PDE Measurements

AFML course project · Team Deadline Desperados (Sampreeth Jaladi, Samarth, Sarva Sandip, Sai Likhith S)

Explainer written 10 November 2026 from the results in the project repository

The project in one paragraph

We watch a simulated fluid at a few sensors and want to recover its viscosity ν. The standard way is trial and error with a physics simulator: guess ν, simulate, compare with the sensors, adjust, repeat. Each simulator run costs about a second, so the number of runs is the cost. We trained a small neural network (an MLP) to make a good *first guess* of ν in a fraction of a millisecond, then let the usual fitting method finish the job. The question is whether that learned first guess saves enough simulator runs, compared with strong classical methods and after paying for training, to be worth it.

The short answer so far (validation data)

- Every method ends up equally accurate; with noisy data the data, not the method, limits accuracy.
- The learned first guess clearly speeds up the fitting method compared with starting in the middle of the range.
- Against the strongest classical method (Brent's search) the saving is small and depends on how precisely the fit is asked to stop: 15–35% less compute with a looser stopping rule, about zero on average with a stricter one.
- The proposal's 2× speedup target was not reached. The pre-registered test-set numbers are still pending.

1. The physical problem

We use the **one-dimensional viscous Burgers equation**, a standard simplified model of fluid flow. A velocity profile u(x, t) moves along a loop (x from 0 to 1, with the ends joined, i.e. periodic boundaries). Faster parts of the wave catch up with slower parts and form steep fronts, like small shock waves. **Viscosity** smooths those fronts and makes the whole wave decay. That is the clue we use: the same starting wave looks different later depending on ν.

$$u_t + (u^2/2)_x = (\nu / \pi)\, u_{xx}$$

The ν/π factor follows the convention of PDEBench (Takamoto et al., NeurIPS 2022), the benchmark whose solver we reproduce. So ν between 0.01 and 0.1 means an effective diffusion coefficient between about 0.0032 and 0.032. Reading PDEBench's source code to confirm this was one of the first steps: the textbook form without π would have made every number in the project wrong by a factor of 3.14.

What we know and what we observe

- **Known:** the starting wave, a sum of three sine waves $u_0(x) = \Sigma\, a_k \sin(2\pi k x + \varphi_k)$ for k = 1, 2, 3, with amplitudes a_k between 0 and 1 and random phases φ_k.
- **Observed:** the velocity at up to 16 fixed sensors (x = 1/32, 3/32, …, 31/32) at 4 times (t = 0.25, 0.5, 0.75, 1.0). Variants use 8 or 4 of those sensors and add Gaussian noise of 1% or 5% of the signal's RMS.
- **Unknown:** the viscosity ν, somewhere in [0.01, 0.1].

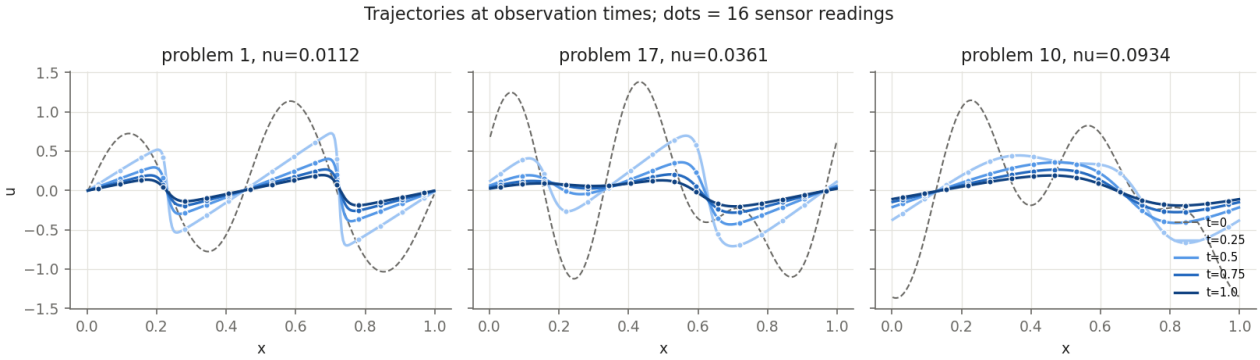


Figure 1. Three example flows (low, middle and high ν). Dashed: the known start. Coloured lines: the flow at the four observation times. Dots: the 16 sensor readings, which are all an estimator gets to see. Low ν keeps sharp fronts; high ν flattens the wave quickly.

2. The simulator (forward model)

Every method, learned or classical, needs to answer the question *"if ν were this value, what would the sensors read?"*. That is the simulator's job. We wrote our own NumPy version of PDEBench's numerical scheme rather than copying its code, because PDEBench's licence restricts redistributing modified code.

How it works

- The loop is split into 256 equal cells; the solver tracks the average velocity in each (a **finite-volume** method).
- At each step it computes how much velocity flows across every cell boundary: a wave-transport part (Rusanov flux with a slope limiter, second-order accurate) and a diffusion part (ν/π times the local slope).
- Time advances in small explicit steps (a two-stage midpoint method). The step must be short enough for stability: at most 0.25 × cell width / max speed, and at most 0.125 × cell width² / (ν/π) for diffusion.
- Sensor readings are taken by linear interpolation between the two nearest cells.

Why runs cost more at high ν

The diffusion limit shrinks the time step in proportion to 1/ν. Simulating to t = 1 took 1,686 steps (0.08 s) at ν = 0.01 but 16,692 steps (0.84 s) at ν = 0.1. So a method's cost depends on *which* ν values it tries, not only on how many. We therefore measure cost in solver steps, not just in runs.

How we know the simulator is right

Test	What it checks	Result
Cole–Hopf exact solution	Our output against an exact mathematical solution of Burgers' equation	Error shrinks as the grid is refined (64 → 128 → 256 cells); below 1% at 256
Heat-equation limit	Tiny waves must decay at the rate set by ν/π	Matches; the textbook ν convention is explicitly rejected
Shift symmetry	Shifting the start shifts the solution exactly (periodic boundaries work)	Exact to 1e-12
Mass conservation	Total momentum must not change	Conserved to 1e-13
Grid convergence at sensors	Readings with 128/256/512 cells against 1,024 cells	Second-order convergence

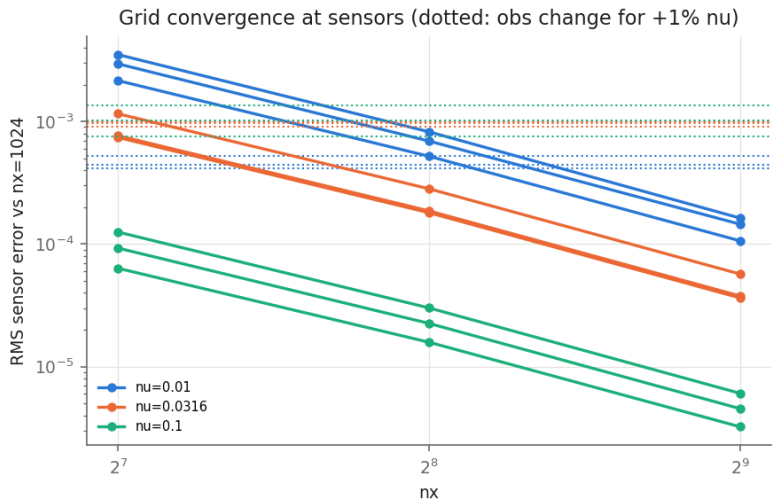


Figure 2. Error at the sensors against a fine 1,024-cell reference. Dotted lines: how much the readings change when ν changes by 1%. At ν = 0.01 the 256-cell error is about as large as a 1% change in ν, so data from a finer grid would bias low-ν estimates by roughly 1–2%; at higher ν the error is negligible.

3. The inverse problem: going from sensors back to ν

To estimate ν we search for the value whose simulated readings best match the observed ones. We search over θ = log ν rather than ν itself, because the range spans a factor of ten and log-space treats every scale fairly. The quantity minimised is the squared mismatch over the available readings:

$$J(\theta) = \tfrac{1}{2} \Sigma \text{ (simulated reading} - \text{observed reading)}^2$$

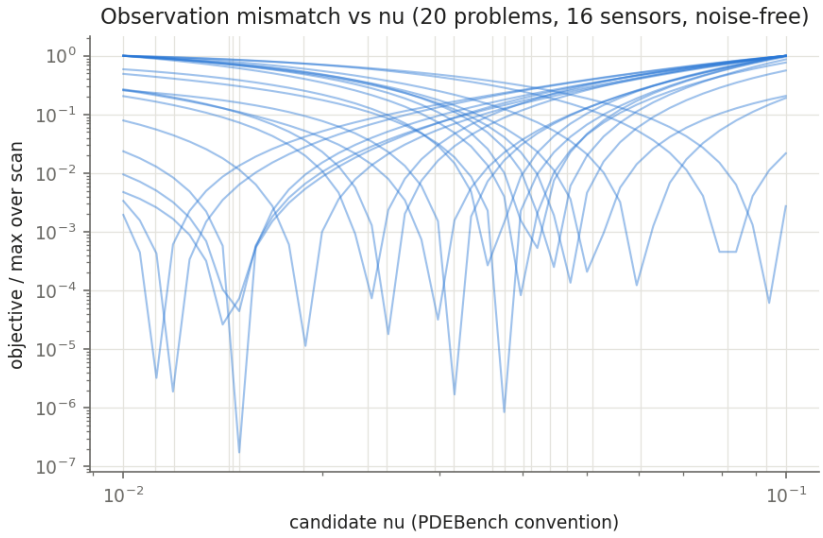


Figure 3. The mismatch J against candidate ν for 20 test problems (no noise, 16 sensors). Each curve has one clear valley at the true ν, so the problem is well-posed and any sensible search will find the answer.

Is the answer always findable?

We checked the shape of J in all 160 problem/condition combinations of the noise pilot: 0 had more than one valley. Every method reached the same best fit in every case, so there were no optimizer failures. When estimates are off, it is because the data carry too little information, not because the search went wrong.

Sensors	Noise	Best possible error (median / 90th pct.)	Brent runs	Midpoint+LSQ runs	Perfect start runs
16	1%	0.26% / 0.8%	10	11	6
16	5%	0.46% / 2.2%	9	11	6
8	1%	0.43% / 1.6%	9	11	6
8	5%	1.89% / 6.0%	9	10	6
4	1%	0.53% / 1.6%	10	11	6
4	5%	2.64% / 13.5%	10	10	8

Table 1. Noise pilot (20 problems per row). "Best possible error" is the error of the best fit any method found; no method can do better with that data. Fewer sensors and more noise make ν harder to pin down. The last column is a diagnostic that starts the fit at the true ν: even then, about 6 runs are needed once noise is present.

4. Our method: a learned first guess

The core idea fits in one picture. The machine-learning part (purple) is fast but only roughly right; the physics-based fit (teal) is precise but needs simulator runs. A good starting point should mean fewer runs.

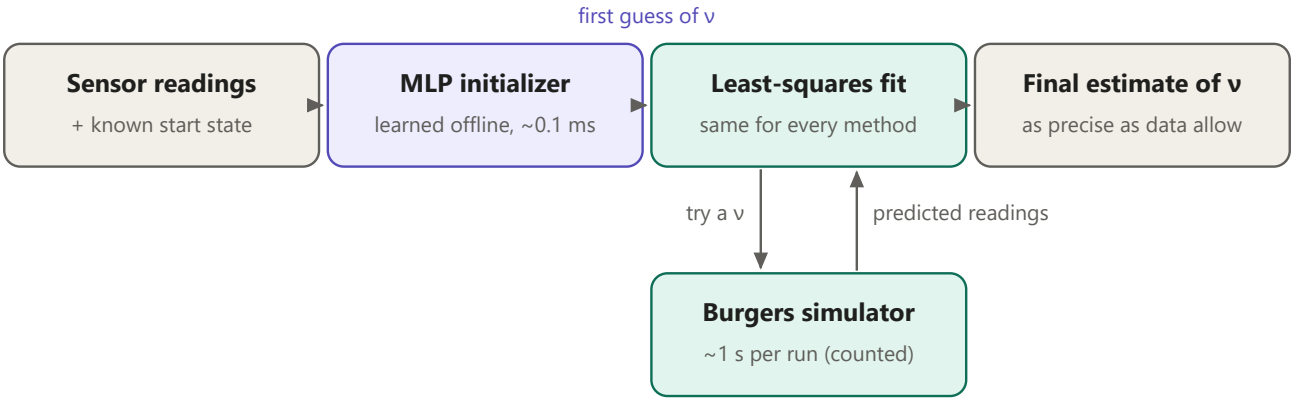


Figure 4. How an estimate is produced. Only the starting point differs between methods; the fit, the simulator and the stopping rules are identical for all of them.

4.1 The training data

- We generate every example with our simulator: pick ν (spread evenly on a log scale), pick a random starting wave, simulate, read the sensors. No external dataset is used; the data are synthetic because the true ν must be known.
- Sizes: 3,000 training, 50 validation and 50 test flows. Each flow has 9 versions (4/8/16 sensors × 0/1/5% noise).
- Splits are made **by flow**: no version of a test flow is ever seen in training. Nine automatic integrity checks enforce this (9 of 9 pass), and every flow regenerates exactly from its seed.

4.2 What the network sees and predicts

Input part	Size	Meaning
Sensor readings	64	16 sensors × 4 times, standardised with training statistics; missing ones set to 0
Availability mask	64	1 if that reading exists, 0 if the sensor is off, so the network knows what is missing
Starting-wave coefficients	6	$a_k \cos \varphi_k$ and $a_k \sin \varphi_k$ for the 3 modes
Output	1	log ν, mapped back to ν and clipped to [0.01, 0.1]

4.3 The network and its training

- A multilayer perceptron: 134 inputs → 64 → 64 → 1, ReLU activations, mean-squared-error loss, Adam optimiser.
- Early stopping on the validation set; a small tuning budget of 4 settings, chosen on validation only (selected: learning rate 0.001, L2 penalty 0.001).
- With 300 training flows it overfit badly (about 20% median error); a learning curve (75 → 150 → 300 flows: about 39% → 32% → 20%) showed more data would help, so we grew the training set to 3,000.

Starting guess	Median error vs true ν	90th percentile
MLP, seed 0	8.2%	32.9%
MLP, seed 1	9.8%	28.8%
MLP, seed 2	11.6%	33.1%
Middle of the range (no learning)	57.4%	—

Table 2. Accuracy of the first guess alone on validation data. The network is far better than a blind guess but about 40× less precise than the data allow (Table 1), which is why the fitting step is still needed.

5. The refinement step and the competitors

5.1 The shared least-squares fit

Starting from any guess, SciPy's bounded least-squares solver (trust-region reflective) improves θ step by step. Each step needs the mismatch at the current θ (one simulator run) and its slope, estimated by nudging θ slightly (a second run). It stops when the step becomes smaller than a tolerance (xtol). A good start means fewer steps.

5.2 Who we compare against

Method	How it chooses where to look	Why it is in the study
Brent's bounded search	No starting guess. Narrows the interval [0.01, 0.1] using golden-section and parabola steps.	Strongest classical method for a one-number problem; the main rival
Midpoint + fit	Starts the shared fit at the middle of the range	What happens with no learning at all
Coarse grid + fit	Tries 5 values first, starts the fit at the best	Classic “scan, then refine”; the 5 runs are counted
kNN + fit	Averages ν of the 10 most similar training cases (same sensors, same inputs as the MLP)	A learned but non-neural starting point: does the network add anything?
MLP + fit (ours)	Starts the shared fit at the network's guess	The method being tested
Oracle + fit	Starts at the true ν	Diagnostic only, never a real method: the best any starting point could do

5.3 Keeping the comparison fair

- Same simulator, same ν bounds, same cases and same stopping tolerances for every method.
- Every simulator run is counted, including the slope runs. Failures would be logged, not dropped (there were none).
- Cost = solver steps × 144.3 µs, the per-step time measured on this laptop. This is used instead of raw wall-clock time, which was unreliable when experiments ran in parallel or the laptop slept.
- Differences are measured on paired cases (the same flow solved by both methods), with 95% confidence intervals from a bootstrap that resamples whole flows, because the 9 versions of one flow are not independent.
- Everything was tuned on validation data. The rules for the test run were written down and committed before testing.
- **Break-even** = offline cost (generating training data + tuning + training) ÷ compute saved per estimate: how many estimates you must make before training pays for itself.

6. Results so far

6.1 How much room is there to save?

Before any machine learning, we measured how quickly each classical method approaches the answer (Figure 5, Table 3), and how much a better starting point helps the fit (Table 4).

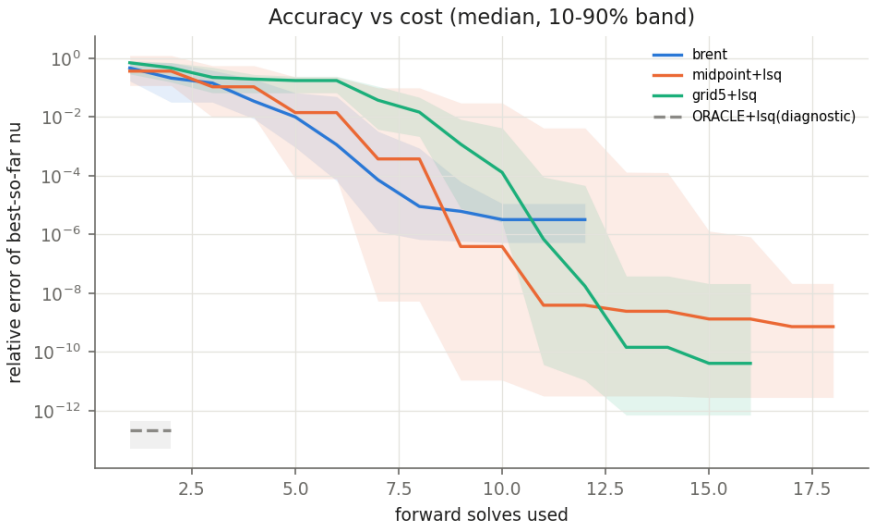


Figure 5. Error of the best estimate so far against the number of simulator runs (20 problems, no noise, 16 sensors; median and 10–90% band). Brent gets close fastest. The oracle's error is exactly zero and is drawn at the bottom.

Method	Runs to 1% error	Runs to 0.1%	Runs to 0.01%	Runs used by its own stopping rule
Brent	5.5	6.5	7	10
Midpoint + fit	6	7	8	11
Coarse grid + fit	9	10	10.5	13
Oracle + fit (diagnostic)	1	1	1	2

Table 3. Median simulator runs needed to reach a given error (no noise). Brent is the strongest classical method.

Starting point	Runs (median)	Worst error
Brent, tolerance 1e-3 (no start)	9	1.7e-04
Fit started 1% off the true ν	6	1.8e-05
Fit started 3% off the true ν	6	8.6e-05
Fit started 10% off the true ν	8	6.5e-05
Fit started 35% off the true ν	8	8.1e-05

Table 4. Starting-point sweep (no noise, matched tolerance). Even a guess within 1% saves only about 3 runs against Brent; a guess within 10%, about what our network achieves, saves about 1. This sets a low ceiling on any speedup.

6.2 The main comparison (validation data)

All five methods were run on the same 450 validation cases (50 flows × 3 sensor counts × 3 noise levels), first with the fit stopping at xtol = 1e-3 (Table 5).

Method	Median error	Median runs	CPU-s per estimate (mean / median / 90th pct.)
Brent	0.25%	9	9.05 / 7.41 / 17.44
Midpoint + fit	0.24%	12	11.86 / 6.73 / 28.30
Coarse grid + fit	0.24%	12	12.29 / 9.89 / 19.86
kNN + fit	0.25%	10	10.13 / 7.01 / 22.98
MLP + fit (ours)	0.24%	8	7.72 / 5.47 / 14.19

Table 5. Validation results, fit tolerance 1e-3. Accuracy is the same for every method; cost differs.

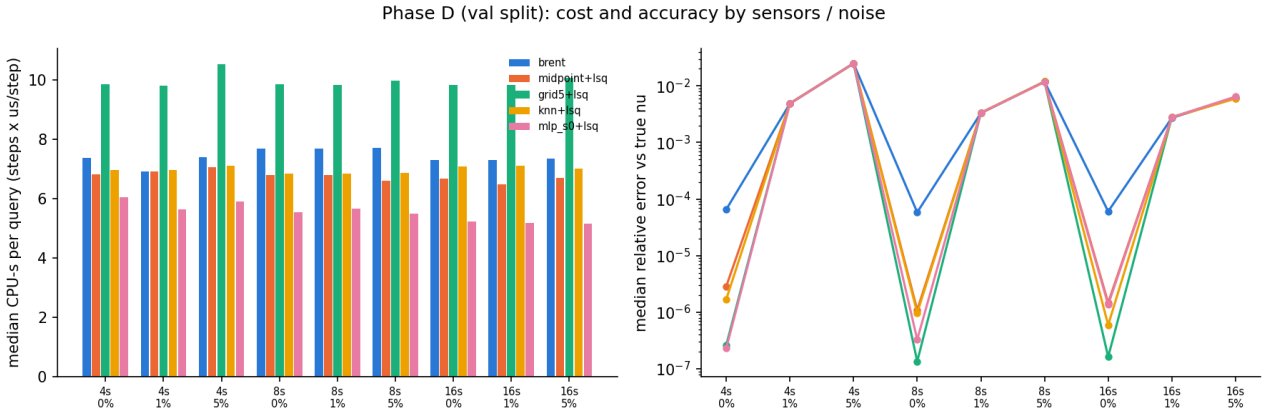


Figure 6. Left: median cost per estimate in each sensor/noise setting. The MLP-started fit (pink) is the cheapest in all nine. Right: median error; in noisy settings all methods coincide because the data limit accuracy.

6.3 Does the saving survive a stricter stopping rule?

Brent and the least-squares fit stop by different rules, so their precision is never perfectly equal. At xtol = 1e-3 the fit is slightly looser than Brent in the hardest cases. We therefore reran everything with the fit 10× stricter (xtol = 1e-4), which makes the fit *more* precise than Brent.

Comparison (paired, CPU-s per estimate)	Fit tolerance 1e-3	Fit tolerance 1e-4
MLP + fit minus Brent: median [95% CI]	-1.49 s [-1.84, -1.16]	-0.65 s [-0.84, -0.22]
MLP + fit minus Brent: mean	-1.33 s	-0.04 s
Break-even against Brent (estimates)	about 1,699	about 64,613
MLP + fit minus Midpoint + fit: median [95% CI]	-2.82 s [-3.51, -1.98]	-2.84 s [-3.58, -1.89]
kNN + fit minus Brent: median [95% CI]	+0.56 s [-0.19, +1.27]	+1.26 s [+0.57, +2.38]

Table 6. Negative numbers mean the first method is cheaper. Break-even uses an offline cost of about 38 CPU-minutes (training data, tuning and training).

7. What the results mean

- **The learned first guess works as designed for the fitting method.** Starting from the network's guess instead of the middle of the range saves about 2.8 CPU-seconds per estimate, consistently, in every setting and at both tolerances.
- **But a good classical search is hard to beat on a one-number problem.** Brent needs only about 9 runs and uses no starting guess. The MLP's advantage over Brent is real at the looser tolerance and at the median, but its mean shrinks to about zero when the fit is made more precise. Part of the earlier saving came from stopping slightly earlier.
- **The ceiling was low from the start.** Even a perfect starting point needs about 6 runs with noise against Brent's 9–10, so no initializer could reach the proposal's 2× target here. The network's guess (about 10% off) captures part of that small gap.
- **The neural network matters, not just "learning".** The kNN guess is much worse (39% median error) and does not save anything against Brent.
- **Accuracy is never sacrificed.** All methods reach the same answer; with noise or few sensors the data, not the method, set the error.

One-line conclusion

A learned first guess reliably makes the fitting method faster, but for a single-parameter problem a good classical search is already so efficient that learning saves little; we measured exactly how little, and when it would pay off.

Limitations to state honestly

- One equation, one unknown parameter, synthetic data. Problems with many parameters may benefit far more from a learned start.
- Validation and test sets are small (50 flows), so confidence intervals are fairly wide.
- Cost is measured as solver steps × a fixed per-step time on one laptop; overhead outside the simulator is ignored (it is tiny).
- Validation data were used to tune the network and kNN, so validation numbers slightly favour the learned methods; the pre-registered test run removes that bias.
- The training data come from the same simulator used for fitting. The planned finer-grid test checks what happens when they differ.

8. What is finished and what remains

Stage	Status
Simulator, correctness tests, classical methods (Phase A)	Done
Noise / fewer-sensor study, starting-point sweep	Done
Dataset with leakage checks (Phase B)	Done
MLP initializer, 3 seeds (Phase C)	Done
Validation comparison at two tolerances (Phase D)	Done
Pre-registered pilot test (3 seeds × 2 tolerances)	Running / pending
Accuracy-versus-cost curves, Cramér–Rao bound, robustness tests, ablations (Phase F)	Not started
Report, slides, submission (Phases G–H)	Not started

Details, commands and task-by-task instructions are in the teammate handover document and in docs/WORKFLOW.md.

Appendix A. Glossary

Term	Meaning
Viscosity ν	How strongly the fluid resists sharp changes in velocity; the number we estimate
Forward problem / simulator	Given ν and the start, compute the flow (and the sensor readings)
Inverse problem	Given the sensor readings, find ν
θ = log ν	The quantity actually searched over, so that every scale is treated evenly
Initializer / first guess	Where the fitting method starts its search
Least-squares fit (refinement)	Iteratively adjusts θ to minimise the squared mismatch with the sensors
Brent's method	A classic search for the minimum of a one-variable function within an interval, no guess needed
kNN	k-nearest neighbours: predicts by averaging the answers of the most similar training examples
Oracle	A diagnostic that cheats by starting at the true answer, to show the best possible case
Validation / test split	Held-out data: validation is used for tuning, test only for the final report
Pre-registration	Writing down the test protocol before looking at test results, so nothing is tuned to them
Bootstrap confidence interval	Uncertainty estimate made by resampling the data many times
Break-even	Number of estimates after which training cost is repaid by the per-estimate saving
Cramér–Rao bound	The smallest error any unbiased method can achieve with given data (planned benchmark)

Appendix B. Where things live in the repository

Path	Contents
blinit/simulator.py	The Burgers solver
blinit/observe.py	Sensor positions, times, sensor masks
blinit/inverse.py	Mismatch function, Brent, least-squares fit, run counting
blinit/data.py	Dataset generation, input features, integrity checks
blinit/model.py, blinit/knn.py	The MLP and the kNN initializers
blinit/experiments/	One script per experiment (evaluate.py, analyze_d.py, …)
configs/	Versioned settings for every experiment
results/	Saved results and figures (figures regenerate without retraining)
tests/	16 automated tests
RESEARCH_LOG.md	Every decision, command and result, in order

Appendix C. Reproducing the main validation result

```
python -m pip install -r requirements.txt
python -m pytest -q
python -m blinit.experiments.build_dataset --config configs/phaseB_v2.json --workers 8
python -m blinit.experiments.analyze_d --dir results/phaseD_v1_val
```

Sources: Takamoto et al. (2022), PDEBench, arXiv:2210.07182; Sambharya et al. (2024), Learning to Warm-Start Fixed-Point Optimization Algorithms, JMLR 25(166); Amos (2023), Tutorial on Amortized Optimization, Found. Trends ML 16(5). The numerical scheme follows PDEBench, developed by NEC Laboratories Europe.